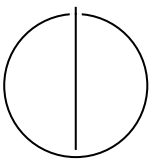
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Adaptation Techniques for using NAS Methods in the FL Setting

Max Coetzee



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

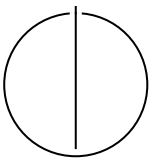
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Adaptation Techniques for using NAS Methods in the FL Setting

Titel

Author:	Max Coetzee
Examiner:	Supervisor
Supervisor:	Advisor
Submission Date:	Submission date



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, Submission date Max Coetzee

Acknowledgments

Abstract

Both *Neural Architecture Search* (NAS) and *Federated Learning* (FL) have made significant progress independently in the past decade. To benefit from the advantages of NAS methods in FL, researchers have started combining them by using NAS in FL [9] [4] [20].

[TODO: overhaul]

Applying techniques from Neural Architecture Search (NAS) to Federated Learning (FL) has been fruitful (remove) in recent years. The combination was identified as a promising research (remove) direction by [14]. It has yielded methods for finding architectures that deal with the challenges imposed by the FL setting.

Research into NAS has grown rapidly [34] since it was popularized by [51]; consequently, literature on its application to FL has grown. The last survey on NAS applied to FL compared approaches of four papers [50]. Since then, we have identified approximately 50 new papers. This motivates a new systematic survey of the landscape to identify progress and gaps in the literature.

In this thesis, we propose a map of the literature landscape based on the FL challenges they address. We achieve this by systematically evaluating the literature and identifying which challenge it solves.

We refer to the FL challenges described in [51], i.e., non-IID data, limited communication, client heterogeneity, privacy of client data, and break them down into smaller subchallenges — each subchallenge being associated with a pattern in the literature. We include personalized FL [31] as an additional subchallenge that was not originally posited, but has since drawn the community's attention.

We then analyze how the subchallenges are addressed and focus on the contribution of the used NAS method towards overcoming the subchallenge. For each subchallenge, we keep track of the NAS types used (following [34], [2]) and assess whether the underexplored methods are candidates for future research.

Neural Architecture Search

Contents

1 Introduction

Deep learning has seen widespread adoption over the last decade and is being applied to an increasingly diverse set of tasks. Applying deep learning traditionally involves a team of deep learning and domain experts who tailor a neural network architecture to a specific task through a lengthy process of trial and error. To automate this process, researchers invented *Neural Architecture Search* (NAS) [6] methods. NAS methods employ diverse strategies to automatically search for a neural network architecture for a given deep learning task within an architecture search space.

Independently, but in parallel to NAS, researchers developed a machine learning approach called *Federated Learning* (FL) [24], which enables training an ML model on data distributed across a set of clients without sharing their data. The model is distributed from the server to the clients, which train the model on their local data. The clients then send the model weight updates to the server, which aggregates them and redistributes them to the clients. This process is repeated for several rounds. FL enhances the privacy of client data and enables training even when clients' data is too large to transfer effectively to a central training site or cannot be shared due to regulatory or privacy constraints.

There has been significant interest in using NAS in FL over the last couple of years, spawning *NAS-in-FL methods* [10], but using NAS methods in FL is not straightforward. Most NAS methods were developed for a centralised setting and can not be applied directly in FL. For example, in a centralised setting, NAS methods typically run on worker nodes connected via low-latency, high-bandwidth links. However, in FL, a NAS method may run on clients connected via an unreliable, high-latency, low-bandwidth network. This creates a *challenge* when transferring the weights of large candidate models between clients and the server to coordinate an architecture search. Specialized solutions are needed to overcome such challenges for NAS-in-FL methods, such as only transferring encodings of candidate architectures to clients [29].

Many solutions for addressing these challenges have been proposed in the NAS-in-FL literature, but solutions are usually bundled as part of individual NAS-in-FL methods. Across the literature, similar challenges and solution mechanisms are described at different levels of abstraction using different terminology. Treating each NAS-in-FL method as an indivisible unit obscures which challenges and solutions recur across the literature at a fine-grained level and does not allow specifying the conditions under

which a specific challenge occurs.

Earlier reviews classify NAS-in-FL methods by properties such as when search occurs and which objectives it optimizes [50], summarize selected methods within broader reviews [16], or discuss NAS as part of multi-objective FL [8]. More recent work supports a more fine-grained comparison through a generic NAS-in-FL pipeline, design elements, and method archetypes [10]. These contributions organize complete methods and their configurations, but reasons for why challenges occur and under which conditions they occur remain underspecified. Challenges remain scattered across the literature along with the recurring solution mechanisms that address them.

Challenge and solution themes alone do not establish their applicability because NAS-in-FL methods operate under different FL conditions and distribute architecture search in different ways. For example, the same client constraints have different consequences depending on whether clients train a complete supernet or evaluate individual candidates. Recurring combinations of FL system characteristics and NAS design properties can therefore be consolidated into common NAS-in-FL scenarios. Relating these scenarios to challenge and solution themes preserves the conditions under which the literature supports each relationship.

NAS-in-FL research lacks a synthesis of challenges, the conditions under which they occur and the solutions used to address challenges. Such a synthesis would (TODO: too strong statement, more like would start shedding light on or begin explainging) clarify why challenges arise, which solution mechanisms the literature proposes for them, and under which conditions those mechanisms (TODO: too strong statement) are applicable. This leads to the following research question:

How do solutions address challenges for common NAS-in-FL scenarios in the literature?

To tackle our research question, we conduct an extensive systematic literature review. We collect an initial set of relevant literature with a Scopus [5] search and extend it with a snowball search. We first use open coding to identify, from the literature, assumption discrepancies and resulting challenges, when using NAS in FL. Next, we extract FL system characteristics and use axial coding to code the influence of characteristics on the relevance of challenges. Then, we extract techniques through open coding and iteratively refine them to synthesise a coherent set of mutually exclusive and collectively exhaustive techniques. Finally, we use axial coding to map techniques onto challenges they overcome.

Our review aims to synthesise techniques for using NAS in FL from the literature to make finding suitable techniques easy in practice. We make the following three contributions. First, by providing a consolidated overview of the challenges of using NAS in FL, practitioners can grasp which issues they can expect to encounter at a glance. Second, by describing how FL system characteristics influence the relevance of challenges, we enable practitioners to focus on the challenges relevant to them. Third,

by extracting techniques for using NAS in FL and highlighting the challenges they address, we enable practitioners to compare existing techniques for using NAS in FL for their specific set of challenges.

In Chapter **??**, we cover the background required for this thesis and related work. In Chapter 2, we describe the method with which we perform our literature review and give an overview of the included literature. In Chapter **??**, we highlight the challenges with FedNAS and how they arise from different FL settings. In Chapter **??**, we describe the adaptation techniques we synthesised and how they overcome challenges. In Chapter 5, we conduct a discussion about our work. Chapter 6 contains our conclusion.

1.1 Background

1.1.1 Neural Architecture Search (NAS)

Neural architecture search (NAS) automates the design of a neural network architecture. There are many different ways to perform NAS, resulting in different *NAS methods*. A NAS methods consists of a search space, a search strategy, and a performance-estimation strategy [6, 34].

The search space defines the potential candidate architectures. Search spaces can be cell-based, ... or ... .

including their operations, connections, depth, and width. The search strategy uses observations gathered during the search to select further candidates, while performance estimation approximates how candidates would perform after full training. A search can optimise predictive performance alone or add objectives such as model size, latency, energy consumption, and communication cost.

Search strategies differ in their required work and state. Black-box strategies select discrete candidates and use measured or predicted performance to guide subsequent selections [34].

One-shot strategies represent many candidates within a shared supernet, from which subnetworks inherit weights. Differentiable variants maintain continuous architecture parameters, whereas other variants sample or progressively prune subnetworks. Weight sharing reduces search cost but introduces a large shared state and can make inherited-weight performance an unreliable proxy for independently trained performance [34].

Performance estimation further changes the cost of either strategy. Full training provides high-fidelity evidence at substantial cost; shortened training and predictors trade estimation fidelity for speed [34]. Together, the three NAS components determine which computations clients perform, what search state they store and exchange, and how frequently they evaluate architectures.

1.1.2 Federated Learning (FL)

Federated learning (FL) enables clients to train a model collaboratively while keeping their raw data local. In a common server-coordinated process, the server selects clients and distributes the global model. Each selected client trains the model locally and returns an update, which the server aggregates for the next communication round [24]. The server consequently receives data-dependent signals; additional privacy mechanisms remain necessary when those signals are sensitive [14].

FL systems vary in scale and client characteristics. Cross-device FL commonly involves many intermittently available mobile or Internet of Things devices, of which only a subset participates in each round. These clients have limited and heterogeneous computation, memory, energy, and wide-area bandwidth [14].

Cross-silo FL usually involves fewer, better-resourced organisations or data centres with stable identities and reliable participation [14]. Its data can be partitioned horizontally, with clients holding different samples, or vertically, with parties holding different features for overlapping samples. In both settings, data quantities and distributions can differ between clients. These settings describe characteristic configurations rather than an exhaustive dichotomy, but establish different expectations about resource constraints, reliability, coordination, and trust.

1.1.3 NAS in FL

NAS in FL incorporates architecture search into the federated process without centralising client data. Conventional NAS training becomes local client training, while architecture instantiation, evaluation, search-state updates, and selection can occur at the server, at clients, or at both [10]. Clients may exchange model weights together with architecture parameters, candidate encodings, controller state, performance scores, or intermediate representations. The search can produce one global model or client- and group-specific models. Candidate quality also depends on its federated training configuration, coupling architecture search with choices such as local epochs and client participation [10].

The search strategy determines how strongly this additional work affects an FL system. Full-supernet gradient and pruning searches impose high per-round computation and memory demands and require a common supernet representation. Subnet-gradient searches reduce individual client work but must reconcile updates to different parts of the supernet. Population- and controller-based searches place smaller candidates on clients and support parallel evaluation, although repeated evaluations can increase total computation and communication [10]. Surrogate evaluation and weight inheritance reduce this cost while making candidate rankings depend on proxy evidence [34, 10].

Challenge applicability therefore follows from the combination of FL system and NAS characteristics. Limited resources, dropout, and unequal search participation are especially severe when a one-shot search runs across many cross-device clients. A cross-silo system may make supernet training feasible, while vertically partitioned data introduces end-to-end coordination and intermediate-representation exchanges [14, 10]. Security concerns apply across settings, although the exposed search signal and the parties able to observe it depend on the chosen search.

Solutions inherit the same context dependence: a mechanism that reduces the size of discrete candidate messages does not address a full-supernet memory limit, while aggregation of architecture parameters presumes a compatible shared representation. The challenge–solution analysis therefore considers both the FL system characteristics and the NAS design to delimit where each challenge and solution applies.

2 Method

We conduct a literature review following [**webster_watson_literature_review_2002**] and analyse the literature on the basis of thematic analysis. I

2.1 Literature Selection

Following [**webster_watson_literature_review_2002**], we combined a database search with backward and forward citation searching. We searched the titles, abstracts and keywords of English-language publications indexed by Scopus using the query `"federated learning" AND "neural architecture search"` [5]. A publication was eligible when it described a method in which federated client work or information contributed to candidate training or evaluation, search-state updates or architecture selection. We excluded publications that addressed only NAS or FL independently and publications in which centrally completed NAS merely preceded FL without performing NAS within FL. Review publications were excluded from the analysis corpus and considered as related work.

For inclusion in the thematic analysis, a publication had to provide evidence from which at least one relevant challenge or solution could be coded. A challenge met this threshold when the publication connected a condition of FL to a consequence for architecture search. A solution met it when the described procedure contained a deliberate intervention whose effect on the NAS-in-FL process could be identified. General claims that an approach addressed concerns such as heterogeneity, efficiency or privacy without explaining the relevant consequence or intervention did not meet this threshold.

We extended the initial set through backward and forward citation searching of the eligible publications, as recommended by [**webster_watson_literature_review_2002**]. Newly identified publications were screened using the same criteria, and citation searching ended when an iteration yielded no additional eligible publications.

TODO: include prisma chart to reduce prose significantly (applicable with webster and watson?)

2.2 Literature Analysis

- analyzed literature using inductive thematic analysis as described in [**using_thematic_analysis_2006**] - for familiarisation we read papers - for initial codes we used the concept matrix proposed by [**webster_watson_literature_review_2002**] and create a row for each NAS-in-FL method where we code all passages that a) indicate FL system characteristics, b) indicate the NAS method type, c) show challenges that are due to using NAS in FL and d) solutions for these challenges

TODO: read what we did in the split and merge rounds of markdown files to flesh this method description out and map it onto thematic analysis - then we iteratively refined challenges and solutions until internal homogeneity and external heterogeneity as described in ...: - we do multiple iterations of merging, splitting up and clarifying boundaries between challenges and solutions as well discarding challenges/solution which make weak claims or are not relevant to our research problem as recommended in ... - challenges are int. hom. and ext. het. when they make just one analytical claim relavant to our narrative - solutions are int. hom. and ext. het. when they consist of just a single mechanism that ... - we then attach solutions to challenges based on (how do we decide if a solution belongs to a challenge? TODO: come up with approach)

- next we derive relevant FL system characteristics and NAS method types - TODO: How did we decide which characteristics to include and how to classify NAS method types?

3 Related Work

Existing reviews and conceptual frameworks have made important contributions to organizing the growing body of research on NAS in FL. Their classifications explain broad procedural differences between approaches and, more recently, the design decisions from which these differences arise. However, their primary unit of analysis is the complete NAS-in-FL method or its design configuration. The challenges that motivate individual design decisions and the solution mechanisms shared across methods consequently remain distributed throughout the primary literature.

Earlier reviews organize NAS-in-FL research around broad procedural and optimization categories. One review distinguishes online from offline implementations and single-objective from multi-objective searches, thereby showing when architecture search occurs and which objectives guide it [50]. A broader review compares NAS and hyperparameter optimization in centralized and federated environments and summarizes how individual approaches modify conventional NAS components for FL [16]. Research on multi-objective methods in FL offers an adjacent perspective by considering architecture search where it contributes to the joint optimization of predictive performance and system objectives [**multi_objective_methods_in_fl_2025**]. These classifications provide an overview of the available approaches, but they do not consolidate the recurring challenges and solutions embedded within them.

More recent work provides a finer-grained account of NAS-in-FL design. A unified design concept decomposes the NAS-in-FL pipeline into design dimensions, identifies recurring method archetypes and relates their configurations to method traits and suitable FL systems [10]. This representation supplies a common vocabulary for comparing approaches that use different terminology and distribute search activities differently between clients and the server. Its analytical focus remains the configuration of complete approaches, leaving the relationships between particular challenges and reusable solution mechanisms implicit within those configurations.

Reviews of NAS and FL separately provide concepts needed to understand these relationships. NAS research distinguishes the search space, search strategy and performance-estimation strategy [6], while FL taxonomies organize concerns such as heterogeneity, efficiency, security and aggregation [1]. Each describes one side of the combined setting. In NAS-in-FL, their interaction can change the problem. For example, the cost of candidate evaluation becomes a distributed client workload, while

differences between searched architectures can invalidate the shared representation assumed by conventional aggregation [**rafl_2024**, 10]. These adjacent reviews therefore establish relevant context without providing an account of the challenge and solution space created by their combination.

This thesis complements the existing classifications by using recurring challenges and solutions as its units of synthesis. It consolidates descriptions that occur under different terminology, distinguishes solutions by their mechanisms and maps the relationships between them. The analysis also retains the FL system characteristics and NAS designs that delimit where a challenge arises and where a solution is applicable. This concept-centric perspective explains why particular changes to conventional NAS are needed and makes visible how the same solution can recur across otherwise different NAS-in-FL methods.

4 Challenges and Solutions for Using NAS in FL

Our analysis revealed five overarching themes of related challenges: *Resource Burdens of NAS in FL*, *Client Heterogeneity*, *Federated Search-State Coordination*, *Disrupted Architecture Search* and *Architecture Search Security*.

Each challenge (TODO: don't let method leak here?) expresses one central analytical claim, although several different solution mechanisms may address it. Each solution remains with the primary challenge that it addresses, while the subsections identify recurring patterns within the broader themes.

If challenges or solutions are reported only for certain FL system characteristics or NAS search strategies or search spaces, then we qualify them accordingly.

- FL system characteristics for which the challenges are applicable/more severe - NAS types based on NAS in FL and NAS 1000 papers categorisation

4.1 Resource Burdens of NAS in FL

NAS may train or estimate the performance of many candidates, while distributing this work across clients adds resource costs to an already expensive procedure [**ying2019nasbench101reproduci** 14]. This theme covers challenges that make federated architecture search operationally burdensome and solutions that reduce, reuse or reorganise the required work.

4.1.1 Computation and Search Effort

Challenge 1.1: Client Resource Limits Make Search Workloads Infeasible

NAS-in-FL becomes infeasible when an assigned search workload exceeds a client's computational, memory or per-round time budget. Client resource constraint can be compounded by hardware heterogeneity, because a workload that is feasible for one client may exceed another client's capacity [**supernet_trade-off_fednas_2025**, 4, 43, 7, 37]. The problem is particularly acute for differentiable supernet-based strategies that distribute a shared supernet and require clients to update several mixed candidate operations locally [**dp-fnas_2020**, 9, 27, 35]. Other strategies usually assign one discrete

or generated candidate at a time, but even one candidate can exceed the resources available to a weak client. This claim concerns whether the assigned work is feasible. Differences in completion time among clients that can perform the work are treated in Challenge 4.2.

Solution 1.1a: Use resource-efficient search components. Practitioners can restrict the search space of candidate architectures to inexpensive components. Prominent examples include depthwise-separable convolutions and mobile-oriented CNN blocks [35, 28]. This designs resource efficiency into the search space, but may exclude expensive components that increase prediction accuracy. Server-side empirical screening of candidate modules is treated separately in Solution 5.1a.

Solution 1.1b: Dispatch capacity-matched subnets from a server-held supernet. During search, the server can retain the complete supernet and assign each client a feasible subspace or sampled subnet instead of a weighted mixture of all candidate operations. The local depth, width or optimisation budget can be adjusted to the client's computation and memory [**apons_2023**, **divide-and-conquer_fednas_2023**, **supernet_trade-off_fednas_2025**, **rafl_2024**, 7, 4, 37, 41, 45, 33, 19, 39]. This keeps the largest search structure off constrained devices and allows weak and strong clients to contribute. The assigned subnet defines the client's search workload and need not be its eventual deployment architecture. Assigning only part of the supernet can create unequal operation and width coverage, which is addressed separately in Challenge 3.2.

Solution 1.1c: Search trainable adaptation modules over a frozen backbone. When a suitable pretrained model is available, clients can keep its backbone fixed and search a sparse combination of lightweight adaptation modules. Only the selected modules and their architecture variables require local optimisation and exchange, reducing both client computation and update size [**fednaspet_2023**]. The benefit depends on how well the fixed backbone transfers to the clients' tasks.

Applicable cross-cutting solution. Architecture-agnostic distillation in Solution 3.3c can instead let lightweight client knowledge models guide a server-held supernet without requiring clients to execute it directly [38].

Challenge 1.2: Repeated Federated Candidate Evaluation Makes Search Costly

Even when each local evaluation is feasible, comparing architectural alternatives across repeated federated rounds consumes substantial computation and elapsed time. Black-box strategies may train several discrete candidates through separate federated evaluations [42, 49], while weight-sharing strategies repeatedly update an over-parameterised supernet or its sampled subnets [9, 17]. Sequential split-model execution can also leave clients idle while other model segments run [13]. Energy provides an additional measure of this work because local optimisation consumes computation energy and

repeated exchanges consume transmission energy [**f2nas_2024**]. Communication volume itself is treated separately in Challenge 1.5. A distinct post-search training cost applies when the selected architecture cannot reuse search-time weights, as discussed in Challenge 1.3.

Solution 1.2a: Allocate candidate evaluation budgets adaptively. This solution applies to black-box search strategies that compare discrete candidates through independent training, including greedy pruning, evolutionary and genetic algorithms, reinforcement learning and Bayesian optimisation. Similar to multi-armed-bandit AutoML approaches [**automl_book_hpo_2019**], each candidate initially receives only a small budget of federated rounds. Interim evidence then determines which candidates are discarded and which receive further rounds.

For example, [42, 43] report that the best candidate usually becomes distinguishable within the first couple of communication rounds. Their procedures drop the remaining candidates and increase the round budget in later search iterations. Evolutionary searches similarly use a fixed small number of rounds or early stopping to limit candidate evaluation [**gpt4-boosted_pso_fednas_2024**, 48].

Solution 1.2b: Prune the supernet progressively. Supernet-based strategies can periodically remove weak candidate operations from the shared supernet, so fewer alternatives remain stored and trained in subsequent federated rounds [7, 37]. Unlike dynamic candidate evaluation budgets, this changes the search space itself rather than the number of rounds allocated to independently trained candidates. Its defining effect is to reduce later search work. Whether the final model requires separate training depends on how its weights are retained, as addressed by Solutions 1.2e and 1.3a. The savings increase as the search progresses, but the initial rounds still incur the cost of the complete supernet and early pruning can permanently remove useful operations.

Solution 1.2c: Predict candidate performance. Performance prediction is a standard AutoML technique for reducing the number of costly configuration evaluations [**automl_book_hpo_2019**]. In NAS-in-FL, candidate-based strategies can evaluate a small set of architectures and train a predictor to estimate the quality of the remaining candidates [38, 30]. Additional care is required because private and non-IID client data can make architecture rankings client-specific. Predictors can therefore be trained separately for each client [38], or learned collaboratively by aggregating their parameters without transmitting the locally evaluated architectures [21]. This reduces the number of candidates evaluated by clients, but its reliability depends on whether the evaluated architectures represent the relevant parts of the search space.

Solution 1.2d: Evaluate independent candidates or subnets in parallel. Since black-box search strategies generate several distinct candidates per iteration, the candidates can be trained and evaluated concurrently by separate client groups [42, 49, 30].

Supernet-based strategies can instead sample several subnets, allocate each to a client

cluster and merge their updates back into the shared supernet [20]. This potentially reduces elapsed search time when enough clients are available, but comparisons can become biased when the groups evaluate candidates on different data distributions [42].
Solution 1.2e: Reuse search-trained weights for candidate evaluation and deployment. Weight-sharing strategies can train common operation weights and allow a candidate to inherit the subset associated with its architecture [32, 45]. Inherited weights avoid training every candidate independently from scratch during evaluation and can initialise the selected subnet before limited final fine-tuning [36]. The inheritance mechanism applies when final architecture selection remains separate from supernet training, whereas Solution 1.3a makes production of the architecture and its deployable weights part of the search. Weights learned while competing subnets were active may nevertheless provide unreliable rankings or a poor final initialisation.
Solution 1.2f: Guide parallel search agents through loss-adaptive supervision. Several search agents can run in parallel while a supervisor compares their relative losses and adjusts the momentum applied by each agent [**supervised_fednas_2023**]. Unlike ordinary candidate parallelism in Solution 1.2d, the operative mechanism changes each agent's optimisation trajectory. It can smooth and accelerate convergence when parallel resources are available, but requires maintaining several searches.
Solution 1.2g: Use split-training idle periods for local search. Auxiliary input and output components can turn an assigned model segment into a locally executable supernet. During otherwise idle periods, a client can sample and train alternative branches without waiting for the remaining model partitions [13]. The corresponding segment can provide a feature-distillation teaching signal, which applies the transfer mechanism in Solution 3.3c to this local search. The defining scheduling intervention converts waiting time into search work.
Solution 1.2h: Adapt local epochs to an energy–convergence bound. The number of local epochs can be selected in each round from an upper bound that relates loss reduction to client computation and communication costs. This balances additional local work against another cloud–client exchange rather than applying one fixed value throughout the search [**f2nas_2024**]. The bound derives a round-specific allocation instead of evaluating the epoch count as part of the joint architecture and FL configuration in Solution 3.4a. The cited evaluation reports an energy reduction of nearly 50%, although applying the mechanism requires estimates of client energy and latency.

Challenge 1.3: Post-Search Retraining Duplicates Federated Computation

Some NAS strategies use search-time training only to select an architecture and then train the selected model from a new initialisation. In NAS-in-FL, this adds another sequence of client-side epochs and aggregation rounds after clients have already

spent computation and communication on the search [36, 46]. This challenge applies whenever the search produces an architecture without reusable final weights and therefore delays the availability of a deployable model.

Applicable cross-cutting solution. Solution 1.2e reuses the selected subnet's search-trained weights for deployment and therefore reduces the duplicated computation described here [36].

Solution 1.3a: Jointly derive an architecture and its deployable weights. One-stage supernet-based strategies can use the same optimisation process to train model weights and concentrate architecture selection until the search state represents a discrete model with trained weights [7, 12]. Reinforcement-learning strategies over a weight-sharing supernet can similarly turn later search rounds into final-model fine-tuning as their sampling probabilities concentrate [39]. This integrates final model production into the search, while progressive pruning in Solution 1.2b primarily removes alternatives to reduce later search work. The final path may remain undertrained if it received too few updates while sharing weights with competing paths.

Challenge 1.4: Heterogeneous Deployment Requirements Multiply Search Cost

A NAS-in-FL deployment may include devices with different inference-time computation, memory or latency constraints. One architecture sized for the weakest client can underuse stronger hardware, while one sized for the strongest client can be infeasible elsewhere [4]. The search must therefore produce several target-specific outcomes. Obtaining each through an independent federated search repeats client computation and communication, so the total cost grows with the number of device classes or deployment budgets [17].

Solution 1.4a: Train one elastic supernet for constraint-specific deployment. A weight-sharing supernet can expose subnets with different depths, widths or exits and train them within one federated process. Clients with similar inference constraints can receive one architecture per capability tier [4, 44], while finer-grained selection can derive a subnet for each client's hardware limits and validation data [17, 38, 47]. This selection determines the deployment outcome, while the capacity matching in Solution 1.1b determines the workload executed by a client during search. Predictor-guided local selection can avoid further federated training and has reduced training cost elevenfold for 20 targets [17]. Co-training many subnets can, however, cause weight-sharing interference and leave rarely sampled parameters stale [17].

Solution 1.4b: Include deployment-resource costs in the architecture objective. Candidate-based and supernet-based strategies can optimise predictive performance subject to measured deployment costs. Latency or multiply–accumulate operation budgets can make outcomes specific to the hardware profile of each client group [**privacy-preserving_nas_acro**

44, 19, 41]. The objective can also penalise the number of weights or expected bytes transmitted during subsequent federated training [48, 3]. This mechanism changes candidate fitness while retaining the alternatives for search, whereas Solutions 1.1a and 5.1a restrict the available modules beforehand. It exposes the trade-off between predictive performance, local inference constraints and recurring model communication before selecting an architecture.

4.1.2 Communication Burden

Cross-device clients often communicate with the server over capacity-limited wide-area links. Typical wide-area bandwidths of 5–25 Mbit/s are far below the more than 10 Gbit/s available within data centres [37]. The discussion below concerns the communication added by search and by the selected model. Differences between client links are treated under disrupted architecture search, where they affect timeliness, participation and candidate assignment.

Challenge 1.5: Architecture Search Multiplies Communication Load

Fixed-model FL exchanges the parameters of one architecture. NAS-in-FL can add both larger payloads and more frequent synchronisation. A supernet contains weights for mutually exclusive operations and can be much larger than any selected architecture, while differentiable strategies may exchange architecture variables alongside model weights throughout the search [4, 38, 9, 3]. The resulting communication load can exceed client transfer budgets even when each individual architecture is feasible.

Solution 1.5a: Compress numerical search updates. Quantisation represents update entries with fewer bits, while sparsification transmits only selected entries. One differentiable supernet-based approach uses 8-bit stochastic quantisation and retains the largest 20% of entries after clipping [3]. These operations reduce the transferred weight and architecture updates without changing the search variables.

Solution 1.5b: Identify known candidates with compact selectors or genotypes. When clients already hold a compatible master model or trained supernet, the server can identify a candidate using a selector or genotype instead of transmitting its parameters. Clients reconstruct the indicated subnet locally and return its evaluation or updated weights [49, 36, 28]. Unlike structural aggregation in Solution 3.3a, this requires a shared search-space definition and common retained weights.

Solution 1.5c: Synchronise architecture variables less frequently. Architecture variables can be treated as slow variables that are updated and uploaded less frequently than model weights. One approach transmits them every H_α rounds and stops doing so after the architecture has been fixed, while ordinary weight training continues [3].

Solution 1.5d: Aggregate search updates hierarchically. A nearby aggregator can combine the model and architecture updates of several clients before forwarding one result to the remote server. Frequent aggregation then uses local links, while fewer exchanges traverse the wide-area network [20]. In one non-IID CIFAR-10 evaluation, this reduced traffic at 80% target accuracy from 3.29–4.68 GB to 1.64 GB [20].

Challenge 1.6: Selected Architectures Can Exceed Federated Training Transfer Budgets

An architecture selected only for predictive performance or local computation can still contain many parameters that FL must transmit in every subsequent training round. The search decision therefore determines the continuing load on client links, and a model that meets computation and memory budgets may still exceed the transfer budget [48].
Applicable cross-cutting solution. Solution 1.4b includes transmitted model size or expected bytes in the architecture objective, allowing the search to account for this recurring training cost before selection [48, 3].

4.2 Client Heterogeneity

Clients within an FL system can differ in the distributions and prediction tasks that give architecture evidence its meaning. This theme covers how these differences affect architecture suitability, supernet optimisation and whose objective guides a shared search. Hardware differences that constrain search or deployment are treated as resource burdens, while their effects on timeliness and search-state coverage are treated in the corresponding themes.

4.2.1 Client Data Heterogeneity

Challenge 2.1: Architecture Suitability Varies Across Data Contexts

Architecture evidence is distribution-dependent. Differences in class balance and feature distributions can cause clients to favour different operations or rank the same candidates differently [23, 17]. Suitability can also change within a client when its operating regime changes over time [3]. Likewise, an architecture ranked on proxy data may perform differently on private target distributions, whose examples cannot be pooled to construct a representative central proxy [**privacy-preserving_nas_across_fl_iot_2021**]. Evidence obtained in one spatial, temporal or proxy context can therefore select an architecture that is unsuitable in another.

Solution 2.1a: Cluster clients by learned architecture preference. When different data contexts require different architectures, clients can be clustered by the similarity of their locally learned architecture variables and train one architecture per cluster [**electricity_load_forecasting_fl_darts_2023**]. Unlike latency-based or distribution-balancing groups, this mechanism uses structural preference as evidence that clients should share an architecture.
Applicable cross-cutting solution. Resource-feasible, data-representative grouping in Solution 4.5a can make the data used to evaluate each candidate approximate the target population [20].
Solution 2.1b: Condition architectural choices on the current regime. A lightweight gate can infer a latent regime and activate corresponding operator adapters within a shared supernet. Clients still collaborate on common weights and architecture distributions, while the active subpath changes with the observed regime [3]. The available evidence is specific to financial time series, so transfer to other forms of temporal drift remains to be established.
Solution 2.1c: Search directly on private target data. Clients can update sampled architectural paths on their local target data and aggregate the resulting model and architecture variables without transferring raw examples [**privacy-preserving_nas_across_fl_iot_2021**]. This changes the data on which architecture suitability is evaluated and avoids relying on a central proxy, although privacy protection still depends on how the search updates are communicated. Including hardware latency in candidate fitness is treated separately in Solution 1.4b.

Challenge 2.2: Distribution-Dependent Gradient Conflict Corrupts Supernet Evaluation

When client distributions produce conflicting gradients, their updates can interfere within a shared supernet. Weight-sharing then couples the candidate evaluations through contested operation weights, which can destabilise optimisation and make inherited-weight rankings unreliable [38, 17]. This optimisation problem can occur even when all clients would ultimately prefer the same discrete architecture.
Solution 2.2a: Optimise the worst-performing subnets across client partitions. A robust supernet objective can target the candidate with the highest loss on each client partition instead of minimising only average candidate loss. Sampling the smallest, largest and random subnets provides a tractable approximation [17]. By preventing high-loss client–subnet combinations from being dominated by easier combinations, the objective reduces the effect of distribution-dependent interference on shared-weight evaluation.

Challenge 2.3: Aggregation Weights Determine Whose Objective Guides Architecture Search

Clients can hold substantially different numbers of examples. Sample-weighted aggregation gives data-rich clients greater influence over shared weights and architecture decisions, while equal weighting gives each participating client the same influence [9]. NAS-in-FL must therefore determine whether the searched architecture should represent pooled examples or participating clients. This choice concerns the unit represented by aggregation rather than whether a candidate estimate is statistically reliable.

Solution 2.3a: Weight search updates to match the evaluation unit. Weighting complete aligned architecture and model updates by local sample count makes the search represent pooled training examples [9]. Equal client weighting instead makes each participating client the aggregation unit. The coefficients should follow whether performance is evaluated across examples or across clients. Parameter exposure and update age provide the different weighting criteria used in Solutions 3.2a and 4.2c.

Applicable cross-cutting solution. Architecture-agnostic distillation in Solution 3.3c can approximate equal client influence by uniformly averaging predictions on shared unlabelled data before training sampled subnets [38].

Challenge 2.4: Clients Solve Related but Different Tasks

Client heterogeneity can extend beyond different distributions for one prediction task. Related local tasks may require different label mappings or structural choices, so a shared architecture and one set of weights can produce incompatible predictions even when every client has sufficient resources [11]. Distribution-aware aggregation alone cannot reconcile objectives that require different outputs for the same input.

Solution 2.4a: Factor the architecture into shared and task-specific components. The search space can separate a base component that captures knowledge shared across tasks from a component that clients adapt structurally. A task-conditioned sampling distribution selects the client-specific component. This factorisation is the architectural personalisation mechanism. Subsequent local fine-tuning adapts the selected component to its task [11].

4.3 Federated Search-State Coordination

FL distributes the variables, model weights and evidence that guide NAS across clients and rounds. This theme covers mechanisms that combine, preserve or jointly optimise that state so the search follows a coherent trajectory.

4.3.1 Coordinating Distributed Search State

Challenge 3.1: Architecture and Model Parameters Form Coupled Global State

Differentiable NAS jointly optimises model weights and architecture variables within a shared supernet topology. The architecture variables determine which candidate operations contribute to the model, while their gradients depend on the current operation weights. Aggregating only model weights discards local structural decisions, whereas combining architecture variables with unrelated weights separates those decisions from the state that produced them [9, 7, 27]. This claim assumes that every client uses the same coordinate system. Structurally incompatible local searches create the different problem described in Challenge 3.3.

Solution 3.1a: Aggregate architecture and model variables together. Clients can return both variable sets after local search, after which the server aggregates and broadcasts the resulting architecture variables with their corresponding model weights [9, 7, 27]. This maintains one shared differentiable search state across rounds. The mechanism determines which coupled variable sets move together, while the client-level influence assigned to them remains the separate choice in Solution 2.3a. Parameter-wise aggregation requires clients to use the same supernet topology and assign the same meaning to every architecture variable.

Challenge 3.2: Resource-Dependent Subnet Assignment Creates Unequal Search-State Coverage

Assigning only hardware-feasible subnets allows clients with different capacities to contribute, but it also determines which parts of a supernet each client can update. Operations and widths that appear only in larger subnets may receive evidence from a small group of capable clients, while unselected parameters receive no local update [**supernet_trade-off_fednas_2025**, 4, 37]. Treating every parameter as though it had been trained uniformly can then distort the shared search state.

Solution 3.2a: Aggregate parameters according to their actual exposure. The server can aggregate each operation only from clients that updated it and weight those updates by the number of examples that passed through the operation [**supernet_trade-off_fednas_2025**, 4, 41]. The resulting coefficients are parameter-specific and reflect exposure rather than the evaluation unit represented by a complete client update. This prevents untrained parameters from being treated as ordinary client updates and records the evidence available for each part of the supernet.

Challenge 3.3: Locally Searched Architectures Can Be Structurally Incompatible

Independent local searches can return discrete graphs with different connections, operations or blocks. Their parameters and architecture variables do not occupy a uniform coordinate system, so ordinary parameter or gradient averaging can lose structural information [**multi-granular_fednas_2023**, 33]. Aggregating only the components shared by several local architectures also discards knowledge learned by their non-overlapping components [**f2nas_2024**].

Solution 3.3a: Exchange and aggregate discrete architecture encodings separately from model parameters. Clients can upload graph encodings without the corresponding model tensors. The server can estimate how often connections and operations occur across local graphs and sample a global graph from the resulting distributions [**multi-granular_fednas_2023**]. A related strategy directly assembles frequently recurring subgraphs [33]. These mechanisms preserve structural information without requiring parameter-wise alignment of complete local architectures. The encoding may contain less data than continuous gradients, but this provides no formal privacy guarantee.

Solution 3.3b: Aggregate compatible blocks by structural similarity. When heterogeneous local architectures share some blocks, the server can aggregate their parameters with weights based on structural similarity [**f2nas_2024**]. This retains direct parameter transfer where a meaningful correspondence exists without treating the complete local architectures as aligned.

Solution 3.3c: Transfer knowledge across architecture boundaries through distillation. Client predictions or feature maps on shared inputs can be combined into a teaching signal for a server supernet, sampled subnets or heterogeneous blocks. The destination architecture can then learn from clients that could not execute it or whose model tensors are incompatible [**f2nas_2024**, 38, 23]. Exchanging compressed predictions similarly enables collaboration between entirely different local topologies [**hetefed_2023**]. This avoids direct tensor alignment, but requires suitable common inputs and transfers output or representation knowledge rather than complete local parameters.

4.3.2 Coordinating Interdependent Search Decisions

Challenge 3.4: FL Control Variables Change Architecture Evaluation

The number of local epochs and the participating-client fraction change the federated trajectory used to obtain candidate fitness. An architecture selected under one FL configuration may therefore rank differently when trained under another [27, 29, 15]. Tuning these control variables only after architecture search can invalidate the comparisons that led to the selected architecture. The challenge is limited to FL control

variables rather than the general interaction between architectures and conventional training hyperparameters.

Solution 3.4a: Search architectures and FL control variables jointly. An alternating procedure can select a training configuration, search an architecture under that configuration and use the result to guide the next configuration decision [27]. Evolutionary and black-box strategies can instead encode the architecture together with variables such as the number of local epochs and the active-client fraction, then evaluate each complete configuration through federated training [29, 15]. Candidate fitness then reflects the FL procedure with which the architecture will operate. This joint fitness search differs from the energy–convergence bound in Solution 1.2h, even when both vary local epochs.

4.3.3 Coordinating Vertically Partitioned Search

Challenge 3.5: Vertical Partitioning Prevents Independent End-to-End Candidate Evaluation

In vertical FL, parties hold different features for the same entities and operate separate parts of a joint model. No feature holder can calculate the complete prediction, loss or architecture gradient because its output must be combined with those of the other parties [22]. This computational dependency is distinct from structural incompatibility between local architectures, which is covered by Challenge 3.3.

Solution 3.5a: Coordinate vertical search through intermediate activations and gradients. Each party can retain its own supernet, weights and architecture variables while participating in a split-model forward and backward pass. Feature-holding parties send intermediate activations to an aggregating party, which computes the joint loss and returns the corresponding gradients. Each party can then update its local weights and architecture variables without requiring compatible local architectures [22]. Unlike distillation in Solution 3.3c, this exchange calculates the joint objective and its exact backward dependencies.

Challenge 3.6: Coordinated Vertical Search Is Restricted to Aligned Samples

The joint forward and backward pass requires records that can be matched across all parties. Their aligned intersection may be much smaller than the union of their local datasets, leaving most observations unable to contribute to the supervised architecture-search objective [22].

Solution 3.6a: Pretrain local search state on unaligned data. Before coordinated search, each party can perform self-supervised one-shot NAS on all of its local observations,

updating both model weights and architecture variables. These states initialise supervised vertical search on the smaller aligned dataset [22]. Unaligned data therefore shape the initial representations and structural preferences, while the joint phase supplies the cross-party supervised objective.

4.4 Disrupted Architecture Search

Client timeliness, availability and structural search dynamics can interrupt architecture search or make its evidence unreliable. This theme distinguishes delays from absence, unequal participation from capability-conditioned candidate assignment and distribution-induced gradient conflict from structural instability.

4.4.1 Participation Disruption and Bias

Challenge 4.1: Client Dropout Interrupts Architecture Search

Cross-device clients can become unavailable because of changing network conditions, battery constraints or competing local workloads. A search procedure that requires the same cohort in every round can then stall or lose updates for the candidates or supernet paths assigned to disconnected clients [45].
Solution 4.1a: Design the search for partial participation. Before each round, the server can select only a random subset of registered clients and aggregate the search updates returned by that subset. The participating clients may change between rounds, allowing architecture search to continue without requiring every registered client to remain available [45]. This planned cohort selection differs from the response threshold in Solution 4.2b, which acts after a round's clients have been selected. Partial participation preserves continuity but does not by itself prevent frequently available clients from shaping the result disproportionately.

Challenge 4.2: Heterogeneous Client Latency Delays or Stales Search Updates

Clients that remain available can nevertheless take different amounts of time to complete assigned search work because their computation and communication speeds differ. A round may wait for the slowest selected client, while updates incorporated after the global search state has advanced may be stale [13, 40, 3]. This timeliness problem is distinct from resource infeasibility, in which a client cannot complete the workload within its budget, and from dropout, in which no update is returned.
Solution 4.2a: Group split-model segments by training latency. Split-model strategies can profile client computation and communication times and form device groups whose

segments have similar completion times. Unlike the data-coverage grouping in Solution 4.5a, this mechanism groups only by observed timing so that supernet training spends less time waiting for the slowest segment [13].

Solution 4.2b: Proceed after a client-response threshold. A soft-synchronous search can advance after most selected clients respond instead of waiting for the entire cohort [40]. The threshold determines when a round progresses after selection, whereas Solution 4.1a determines which clients enter the cohort before the round.

Solution 4.2c: Compensate or down-weight stale search updates. Late updates can be incorporated using delay compensation after the global search state has advanced [40]. Differentiable search can instead reduce an update's aggregation weight as its staleness increases [3]. These mechanisms transform the influence of a returned update according to its age rather than the evaluation unit in Solution 2.3a or parameter exposure in Solution 3.2a.

Challenge 4.3: Personalised Architecture Searches Converge at Different Times

In personalised differentiable search, available clients can optimise architecture variables at different rates because their data distributions and local trajectories differ. A common stopping round can terminate some searches prematurely while allowing others to overfit, for example by increasingly favouring skip connections [23]. This claim concerns client-specific convergence and stopping rather than delayed updates or client absence.

Solution 4.3a: Stop architecture updates independently. After a shared warm-up period, each client can stop updating its architecture variables when a local criterion detects increasing preference for skip connections. Other clients continue searching until they meet their own stopping criteria [23].

Challenge 4.4: Variable Participation Biases Search Towards Available Clients

Differences in bandwidth and availability cause some clients to participate more often or meet more round deadlines than others [45, 37]. Even when partial participation allows the search to continue, the data of frequently available clients can influence shared weights and architecture rankings more often than the data of intermittently available clients. The search can therefore become unrepresentative without being interrupted.

Applicable cross-cutting solution. Resource-feasible, data-representative grouping in Solution 4.5a can be applied when selecting participants so that recurring availability alone does not determine whose data enter the search [26, 20].

Challenge 4.5: Resource-Aware Candidate Assignment Confounds Architecture Evaluation

Resource-aware assignment can make small candidates train mainly on low-capability clients and large candidates on well-connected clients [4]. Under non-IID data, architectural capacity is then systematically coupled with the data of the clients able to execute it. Candidate quality can reflect this assignment pattern rather than the architecture alone, even when every selected client returns an update [38].

Solution 4.5a: Form resource-feasible, data-representative client groups. Client groups can be formed under a completion-time or bandwidth constraint and adjusted so that their combined class distribution better represents the target population. The literature implements this by rebalancing bandwidth-based groups according to class distribution or by minimising each cluster's distance from the population distribution under a completion-time limit [26, 20]. Applying the mechanism separately to each candidate makes comparisons less dependent on which client links support a particular architecture. Applying it to round-level participant selection can also reduce availability-driven representation bias.

Applicable cross-cutting solution. Architecture-agnostic distillation in Solution 3.3c transfers predictions or features into candidates that some clients cannot execute, decoupling client knowledge from direct candidate assignment [38, 23].

4.4.2 Structural Instability

Challenge 4.6: Structural Decisions Can Destabilise Distributed Search

Distributed structural search can become non-stationary when successive architecture updates change which operations or weights participate in optimisation. Layer-wise updates can favour different operations at different depths, and an early preference for parameter-free operations can prevent convergence [18]. Fixing an operation during progressive selection changes the computation graph and its weight-sharing relationships, so the next decision may be evaluated before the remaining weights adapt [25]. Changing subnet coverage can likewise create jagged aggregate updates and slow supernet convergence [**divide-and-conquer_fednas_2023**]. These are structural causes of instability rather than conflicts induced by client data distributions, which are covered by Challenge 2.2.

Solution 4.6a: Regularise layer-wise architecture updates. During local differentiable search, a gradient-alignment term can constrain the relationship between layer gradients and architecture variables. Incorporating this term into the architecture objective counteracts depth-dependent operation preferences before the client update is aggregated [18].

Solution 4.6b: Separate structural decisions with recovery rounds. A progressive supernet search can first warm up the shared weights and then fix one operation at a time according to its effect on local validation accuracy. Several recovery rounds update the remaining weights after each decision before the next edge is evaluated [25]. Recovery between decisions is the defining intervention, whereas progressive pruning in Solution 1.2b removes alternatives to reduce later work. Once the relevant edges have been resolved, the architecture is fixed and ordinary training continues.
Solution 4.6c: Vary subnet coverage over the search. A server can assign subnet masks with large pairwise distances during early rounds and progressively reduce those distances as training advances. Early rounds explore separated regions of the search space, while later rounds concentrate updates around better understood regions [**divide-and-conquer_fednas_2023**]. The structured masks also fit together more consistently during aggregation than independently sampled subnets.

4.5 Architecture Search Security

Architecture search exposes structural signals and decisions in addition to conventional model updates. This theme covers integrity and confidentiality risks created or altered by distributed architecture search.

Challenge 5.1: Poisoning the Architecture Search

Malicious clients can manipulate model weights and architecture-search signals through poisoned data or local optimisation. The attack can therefore pursue a backdoor objective while changing operation probabilities or candidate rankings. In weight-sharing search, compromised weights can also affect every candidate that reuses them. Experiments with one malicious client among eight found substantial degradation in two discrete weight-sharing strategies under fourfold-scaled updates, showing that poisoning can persist in the selected architecture [46].
Conditional observation. In differentiable supernet-based search, each candidate operation remains part of a weighted mixture while clients update both its weights and selection probability. The cited analysis derives a reduction in an operation's mixture contribution when its poisoned weights provide an opposing or comparatively small gradient for the main task. This result assumes Lipschitz continuity, convexity, bounded gradients and a unique global optimum [46]. The evaluation supports this behaviour only for the studied backdoor attacks and search spaces. Retraining the selected fixed architecture from scratch did not preserve the reported robustness, so it is a conditional property of the trained search state rather than a deliberate defence or intrinsic robustness of the architecture [46].

Solution 5.1a: Pre-screen search-space modules on server data. The server can evaluate modules on server-held data and select a compact set that balances predictive performance, memory and latency before clients begin differentiable search. Unlike the use of generally inexpensive primitives in Solution 1.1a, this mechanism empirically screens the available modules against server data. It also differs from Solution 1.4b because screening removes modules before search rather than changing their fitness within the search. Constraining model complexity in this way reduced overfitting to noise introduced by backdoor triggers in the cited evaluation [46]. This evidence is limited to the studied attacks and search setting.

Challenge 5.2: Architecture-Search Artefacts Create Additional Privacy Exposure

NAS-in-FL communicates data-dependent signals that fixed-architecture FL does not require. Differentiable strategies can expose architecture gradients, while other strategies may transmit discrete architecture graphs, personalised choices or scalar rewards [**dp-fnas_2020**, **multi-granular_fednas_2023**, 46, 27]. The cited literature treats local sample counts, class proportions and example reconstruction as possible attack targets rather than demonstrating their recovery [**multi-granular_fednas_2023**]. It also proposes that personalised architecture choices could help identify the source of an update, but does not evaluate such an attack [46]. These data-dependent artefacts therefore constitute a potential privacy exposure whose practical severity depends on the transmitted representation and adversary.

Solution 5.2a: Apply differential privacy to search signals. Differentiable strategies can clip model and architecture gradients and add Gaussian noise before upload, while scalar hyperparameter rewards can be perturbed in the same way [**dp-fnas_2020**, 27]. In vertically partitioned search, clipping and noise can instead protect the intermediate activations and backward gradients exchanged between parties [22]. The added noise can, however, impair model and search performance [46].

Solution 5.2b: Securely aggregate aligned search variables. When every client updates the same differentiable supernet, model weights and architecture variables have aligned shapes and can use the same weighted secure-aggregation protocol [46]. This cryptographically conceals individual updates from the server but requires a uniform search representation during collaboration.

Solution 5.2c: Keep personal architecture selection local. After a shared search, clients can select and adapt their personal architectures without transmitting these choices to the server [46]. This conceals each client's deployed architecture independently of the mechanism used to protect the preceding shared updates.

Evidence qualification. The discrete structural encoding exchanged in Solution 3.3a has been argued to limit input reconstruction because its dimension can be much

smaller than the private data. For a seven-layer example, the cited analysis compares a 56-entry derivative matrix with the 50,176 pixels in an image batch and argues that the resulting optimisation has many possible solutions, making precise pixel reconstruction by that procedure impractical [**multi-granular_fednas_2023**]. The smaller encoding is a property of the representation and provides no formal privacy guarantee. It does not exclude property or source inference and may still require differential privacy or encryption.

5 Discussion

- the further the FL system deviates from centralised NAS, the more challenging it is => conversely, when NAS-in-FL degenerates to centralised NAS in some aspects it becomes simpler - show how some solutions limit compatibility with others and how they are - the search space trade-off - suprisingly many NAS-in-FL methods do X - we expected federation scale to play a more prominent role, but this was barely ever considered by NAS-in-FL methods and the largest federation scale currently in the literature is ..., signifcantly smaller than regular FL setups - similar to NAS research, supernet-based methods have become more popular over time - performance evaluation is basically universally just training a candidate architecture for a couple of epochs, making the main differentiating factors for challenge applicability the NAS search strategy type and secondarily the search space - The FL system characteristics which NAS-in-FL methods apply to tend to be underspecified

5.1 Principal Findings

5.2 Implications for Practice and Research

- clear guidance on existing related solutions and challenges and the context/circumstances under which challenges arise

5.3 Limitations

- rely on literature for facts, i.e. described only challenges and solutions from the literature - few NAS-in-FL methods establish an explicit causal link between FL system characteristics and challenges, therefore this needs to be treated as corrolation evidence, not causal

5.4 Future Research

- large share

6 Conclusion

- the amount of possible knobs to turn in NAS-in-FL is astronmical and the literature has only investigated a small subset of them - NAS in FL trails behind NAS and FL - provide some clear direction for future research

Abbreviations

List of Figures

List of Tables

Bibliography

[1] M. Arbaoui, M.-A. Brahmia, A. Rahmoun, and M. Zghal. "Federated learning survey: A multi-level taxonomy of aggregation techniques, experimental insights, and future frontiers." In: *ACM Trans. Intell. Syst. Technol.* 15.6 (Nov. 2024). ISSN: 2157-6904. DOI: 10.1145/3678182.

[2] S. S. P. Avval, N. D. Eskue, R. M. Groves, and V. Yaghoubi. "Systematic review on neural architecture search." In: *Artificial Intelligence Review* 58.3 (Jan. 6, 2025), p. 73. ISSN: 1573-7462. DOI: 10.1007/s10462-024-11058-w.

[3] Z. Chen, H. Zhang, S. Wang, and J. Chen. "FedRegNAS: Regime-Aware Federated Neural Architecture Search for Privacy-Preserving Stock Price Forecasting." In: *Electronics* 14.24 (2025). DOI: 10.3390/electronics14244902.

[4] L. Dudziak, S. Laskaridis, and J. Fernandez-Marques. *FedorAS: Federated architecture search under system heterogeneity*. 2022. arXiv: 2206.11239 [cs.LG].

[5] Elsevier. *Scopus*. Abstract and citation database. Accessed 2026-01-17. 2026.

[6] T. Elsken, J. H. Metzen, and F. Hutter. "Neural architecture search: A survey." In: *Journal of Machine Learning Research* 20.55 (2019), pp. 1–21.

[7] A. Garg, A. K. Saha, and D. Dutta. *Direct federated neural architecture search*. 2020. arXiv: 2010.06223 [cs.LG].

[8] M. Hartmann, G. Danoy, and P. Bouvry. *Multi-objective methods in Federated Learning: A survey and taxonomy*. 2025. arXiv: 2502.03108 [cs.LG].

[9] C. He, M. Annavaram, and S. Avestimehr. *Towards Non-I.I.D. and invisible data with FedNAS: Federated deep learning via neural architecture search*. 2021. arXiv: 2004.08546 [cs.LG].

[10] N. Henze, S. Rank, D. Deng, M. Lindauer, A. Sunyaev, and N. Kannengießer. "Neural Architecture Search in Federated Learning: A Unified Design Concept." In: *TechRxiv* (2026). DOI: 10.36227/techrxiv.176964091.16274890/v1.

[11] M. Hoang and C. Kingsford. "Personalized Neural Architecture Search for Federated Learning." In: *1st NeurIPS Workshop on New Frontiers in Federated Learning (NFFL 2021)* ().

[12] X. Huang and J. Gao. *Federated neural architecture search with model-agnostic meta learning*. 2025. arXiv: 2504.06457 [cs.LG].

[13] C. Huo, J. Jia, T. Deng, M. Dong, Z. Yu, and D. Yuan. "NASFLY: On-device split federated learning with neural architecture search." In: *2024 IEEE international symposium on parallel and distributed processing with applications (ISPA)*. 2024, pp. 2184–2190. DOI: 10.1109/ISPA63168.2024.00298.

[14] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L. D'Oliveira, H. Eichner, S. E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, M. Raykova, H. Qi, D. Ramage, R. Raskar, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, and S. Zhao. *Advances and Open Problems in Federated Learning*. 2021. arXiv: 1912.04977 [cs.LG].

[15] S. Khan, F. Nosheen, S. S. A. Naqvi, H. Jamil, M. Faseeh, M. A. Khan, and D.-H. Kim. "Bilevel Hyperparameter Optimization and Neural Architecture Search for Enhanced Breast Cancer Detection in Smart Hospitals Interconnected With Decentralized Federated Learning Environment." In: *IEEE Access* 12 (2024), pp. 63618–63628. DOI: 10.1109/ACCESS.2024.3392572.

[16] S. Khan, A. Rizwan, A. N. Khan, M. Ali, R. Ahmed, and D. H. Kim. "A multi-perspective revisit to the optimization methods of Neural Architecture Search and Hyper-parameter optimization for non-federated and federated learning environments." In: *Computers and Electrical Engineering* 110 (2023), p. 108867. ISSN: 0045-7906. DOI: https://doi.org/10.1016/j.compeleceng.2023.108867.

[17] A. Khare, A. Agrawal, A. Annavajjala, P. Behnam, M. Lee, H. Latapie, and A. Tumanov. *SuperFedNAS: Cost-efficient federated neural architecture search for on-device inference*. 2024. arXiv: 2301.10879 [cs.LG].

[18] J. Li, S. Wang, R. Yang, M. Gong, Z. Hu, N. Zhang, K. Sheng, and Y. Zhou. "Toward Federated Customized Neural Architecture Search for Remote Sensing Scene Classification." In: *IEEE Transactions on Geoscience and Remote Sensing* 63 (2025), pp. 1–12. DOI: 10.1109/TGRS.2025.3537085.

[19] Y. Li, R. Yao, D. Qin, and Y. Wang. "Lightweight federated learning for on-device non-intrusive load monitoring." In: *IEEE Transactions on Smart Grid* 16.2 (2025), pp. 1950–1961. DOI: 10.1109/TSG.2024.3482363.

[20] J. Liu, J. Yan, H. Xu, Z. Wang, J. Huang, and Y. Xu. "Finch: Enhancing federated learning with hierarchical neural architecture search." In: *IEEE Transactions on Mobile Computing* 23.5 (2024), pp. 6012–6026. DOI: 10.1109/TMC.2023.3315451.

[21] S. Liu, X. Wang, and Y. Jin. "Federated Bayesian Optimization for Privacy-Preserving Neural Architecture Search." In: *2023 IEEE Congress on Evolutionary Computation (CEC)*. 2023, pp. 1–8. DOI: 10.1109/CEC53210.2023.10254155.

[22] Y. Liu, X. Liang, J. Luo, Y. He, T. Chen, Q. Yao, and Q. Yang. "Cross-Silo Federated Neural Architecture Search for Heterogeneous and Cooperative Systems." In: *Federated and Transfer Learning*. Ed. by R. Razavi-Far, B. Wang, M. E. Taylor, and Q. Yang. Cham: Springer International Publishing, 2023, pp. 57–86. ISBN: 978-3-031-11748-0. DOI: 10.1007/978-3-031-11748-0_4.

[23] Y. Liu, S. Guo, J. Zhang, Z. Hong, Y. Zhan, and Q. Zhou. "Collaborative Neural Architecture Search for Personalized Federated Learning." In: *IEEE Transactions on Computers* 74.1 (2025), pp. 250–262. DOI: 10.1109/TC.2024.3477945.

[24] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. "Communication-efficient learning of deep networks from decentralized data." In: *Proceedings of the 20th international conference on artificial intelligence and statistics*. Ed. by S. Aarti and Z. Jerry. Vol. 54. Proceedings of machine learning research. PMLR, Apr. 2017, pp. 1273–1282.

[25] E. Mushtaq, C. He, J. Ding, and S. Avestimehr. *SPIDER: Searching personalized neural architecture for federated learning*. 2021. arXiv: 2112.13939 [cs.LG].

[26] G. Öcal and A. Özgövde. "Network-aware federated neural architecture search." In: *Future Generation Computer Systems* 162 (2025), p. 107475. ISSN: 0167-739X. DOI: https://doi.org/10.1016/j.future.2024.07.053.

[27] J. Seng, P. Prasad, M. Mundt, D. S. Dhami, and K. Kersting. *FEATHERS: Federated Architecture and Hyperparameter Search*. 2023. arXiv: 2206.12342 [cs.LG].

[28] J.-M. Shao, G.-Q. Zeng, K.-D. Lu, G.-G. Geng, and J. Weng. "Automated federated learning for intrusion detection of industrial control systems based on evolutionary neural architecture search." In: *Computers & Security* 143 (2024), p. 103910. DOI: 10.1016/j.cose.2024.103910.

[29] R. E. Shawi. "AgingFedNAS: Aging evolution federated deep learning for architecture and hyperparameter search." In: *Advances in knowledge discovery and data mining*. Ed. by X. Wu, M. Spiliopoulou, C. Wang, V. Kumar, L. Cao, Y. Wu, Y. Yao, and Z. Wu. Singapore: Springer Nature Singapore, 2025, pp. 107–119. ISBN: 978-981-96-8173-0.

[30] J. Su, C. Tang, and Z. Liu. "A Prediction Optimization Method with Federated Learning and Neural Architecture Search for Distributed Renewable Energy Sources." In: *Distributed Generation & Alternative Energy Journal* 40.2 (2025), pp. 213–238. DOI: 10.13052/dgaej2156-3306.4021.

[31] A. Z. Tan, H. Yu, L. Cui, and Q. Yang. "Towards personalized federated learning." In: *IEEE Transactions on Neural Networks and Learning Systems* 34.12 (2023), pp. 9587–9603. DOI: 10.1109/TNNLS.2022.3160699.

[32] C. Wang, B. Chen, G. Li, and H. Wang. "Automated Graph Neural Network Search under Federated Learning Framework." In: *IEEE Transactions on Knowledge and Data Engineering* 35.10 (2023), pp. 9959–9972. DOI: 10.1109/TKDE.2023.3250264.

[33] X. Wei, G. Chen, C. Yang, H. Zhao, C. Wang, and H. Yue. "EFNAS: Efficient federated neural architecture search across AIoT devices." In: *2024 international joint conference on neural networks (IJCNN)*. 2024, pp. 1–8. DOI: 10.1109/IJCNN60899.2024.10650653.

[34] C. White, M. Safari, R. Sukthanker, B. Ru, T. Elsken, A. Zela, D. Dey, and F. Hutter. *Neural architecture search: Insights from 1000 papers*. 2023. arXiv: 2301.08727 [cs.LG].

[35] R. Wu, C. Li, J. Zou, and S. Wang. "FedAutoMRI: Federated neural architecture search for MR image reconstruction." In: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023 Workshops*. Berlin, Heidelberg: Springer, 2023, pp. 347–356. DOI: 10.1007/978-3-031-47401-9_33.

[36] Y. Wu, H. Fan, W. Ying, Z. Zhou, Q. Zheng, and J. Zhang. "Federated Neural Architecture Search with Hierarchical Progressive Acceleration for Medical Image Segmentation." In: *Advances in Swarm Intelligence*. Ed. by Y. Tan and Y. Shi. Singapore: Springer Nature Singapore, 2024, pp. 112–123. ISBN: 978-981-97-7184-4.

[37] J. Yan, J. Liu, H. Xu, Z. Wang, and C. Qiao. "Peaches: Personalized federated learning with neural architecture search in edge computing." In: *IEEE Transactions on Mobile Computing* 23.11 (2024), pp. 10296–10312. DOI: 10.1109/TMC.2024.3373506.

[38] A. Yang and Y. Liu. "Heterogeneity-aware personalized federated neural architecture search." In: *Entropy* 27.7 (2025). ISSN: 1099-4300. DOI: 10.3390/e27070759.

[39] D. Yao and B. Li. "PerFedRLNAS: One-for-all personalized federated neural architecture search." In: *Proceedings of the AAAI Conference on Artificial Intelligence* 38.15 (Mar. 2024), pp. 16398–16406. DOI: 10.1609/aaai.v38i15.29576.

[40] D. Yao, L. Wang, J. Xu, L. Xiang, S. Shao, Y. Chen, and Y. Tong. "Federated model search via reinforcement learning." In: *2021 IEEE 41st international conference on distributed computing systems (ICDCS)*. 2021, pp. 830–840. DOI: 10.1109/ICDCS51616.2021.00084.

[41] W. Ying, C. Wang, Y. Wu, X. Luo, Z. Wen, and H. Zhang. "FGFL: Fine-grained federated learning based on neural architecture search for heterogeneous clients." In: *Advances in Swarm Intelligence*. Singapore: Springer Nature Singapore, 2024, pp. 99–111.

[42] J. Yuan, M. Xu, Y. Zhao, K. Bian, G. Huang, X. Liu, and S. Wang. *Federated neural architecture search*. 2022. arXiv: 2002.06352 [cs.LG].

[43] J. Yuan, M. Xu, Y. Zhao, K. Bian, G. Huang, X. Liu, and S. Wang. "Resource-Aware Federated Neural Architecture Search over Heterogeneous Mobile Devices." In: *IEEE Transactions on Big Data* (2022). DOI: 10.1109/TBDATA.2022.3227403.

[44] C. Zhang, X. Yuan, Q. Zhang, G. Zhu, L. Cheng, and N. Zhang. "Toward tailored models on private AIoT devices: Federated direct neural architecture search." In: *IEEE Internet of Things Journal* 9.18 (2022), pp. 17309–17322. DOI: 10.1109/JIOT.2022.3154605.

[45] F. Zhang, J. Ge, C. Wong, S. Zhang, C. Li, and B. Luo. "Optimizing Federated Edge Learning on Non-IID Data via Neural Architecture Search." In: *2021 IEEE Global Communications Conference (GLOBECOM)*. 2021, pp. 1–6. DOI: 10.1109/GLOBECOM46510.2021.9685909.

[46] F. Zhang, M. Li, J. Ge, F. Tang, S. Zhang, J. Wu, and B. Luo. "Privacy-Preserving Federated Neural Architecture Search with Enhanced Robustness for Edge Computing." In: *IEEE Transactions on Mobile Computing* 24.3 (2025), pp. 2234–2252. DOI: 10.1109/TMC.2024.3490835.

[47] Z. Zhang, Z. Liu, Y. Zhao, C. Qiu, C. Zhang, and X. Wang. "ENASFL: A federated neural architecture search scheme for heterogeneous deep models in distributed edge computing systems." In: *IEEE Transactions on Network Science and Engineering* 11.3 (2024), pp. 2610–2620. DOI: 10.1109/TNSE.2023.3344850.

[48] H. Zhu and Y. Jin. "Multi-Objective Evolutionary Federated Learning." In: *IEEE Transactions on Neural Networks and Learning Systems* 31.4 (2020), pp. 1310–1322. DOI: 10.1109/TNNLS.2019.2919699.

[49] H. Zhu and Y. Jin. *Real-time federated evolutionary neural architecture search*. 2020. arXiv: 2003.02793 [cs.LG].

[50] H. Zhu, H. Zhang, and Y. Jin. "From federated learning to federated neural architecture search: a survey." In: *Complex and Intelligent Systems* 7.2 (Apr. 2021), pp. 639–657. ISSN: 2198-6053. DOI: 10.1007/s40747-020-00247-z.

[51] B. Zoph and Q. V. Le. *Neural architecture search with reinforcement learning*. 2017. arXiv: 1611.01578 [cs.LG].